Data Science & AI

DATA STRUCTURES Through Python

Linked Lists

Lecture No.- 01

By- Satya sir

Topics to be Covered

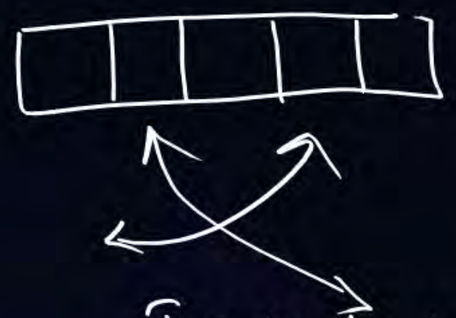
- Linked Lists DS
  - what is LL?
  - Types of LL
  - operations
  - Construct a Linked List

Topic : Linked Lists – Operations, Types

Data Structure :

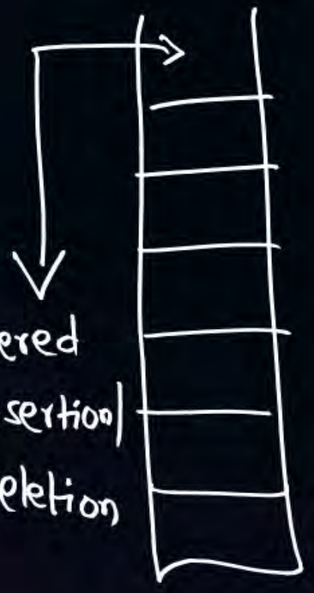
It defines the way to organize / represent data

Array DS



Insertion / Deletion at random locations

Stack DS



Ordered Insertion / Deletion

Queue DS

Topic : Linked Lists – Operations, Types

Linked List Data Structure

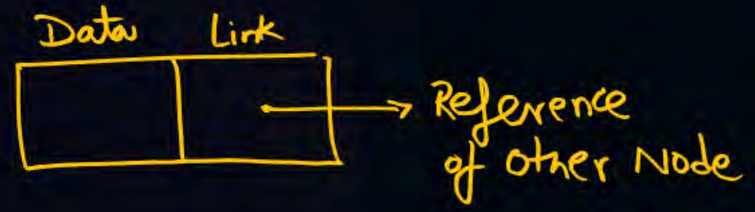
- A list of Elements (or) Nodes, that linked to other Node/Nodes.
- Elements of Linked list are assigned memory at random locations.
- Types of Linked Lists :
  1) Singly Linked List (SLL)
  2) Doubly Linked List (DLL)
  3) Circular Linked List – Singly / Doubly

Topic : Linked Lists – Operations, Types

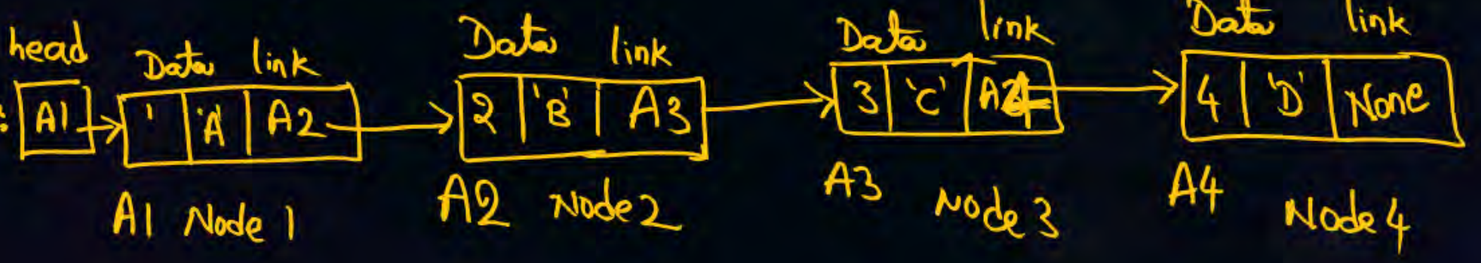
Singly Linked List (SLL)

- Every Node of SLL will have 2 Parts
  1) Data Part
  2) Link Part / Reference to other Node

SLL Node :



SLL with multiple Nodes :



Last Node of SLL

- A Node whose link maintain the reference as None.

First Node of SLL

A Node called head maintain reference of First Node.

Topic : Linked Lists – Operations, Types

Operations on Linked Lists

- Creation of a Node
- Construction of a list with multiple Nodes
- Insertion (or) Add a Node to the list
- Deletion (or) Removal of a Node from the list
- Search for an Element in the List
- Sort Elements of Linked List
- Traverse (or) Access Elements of Linked List

Topic : Linked Lists – Operations, Types

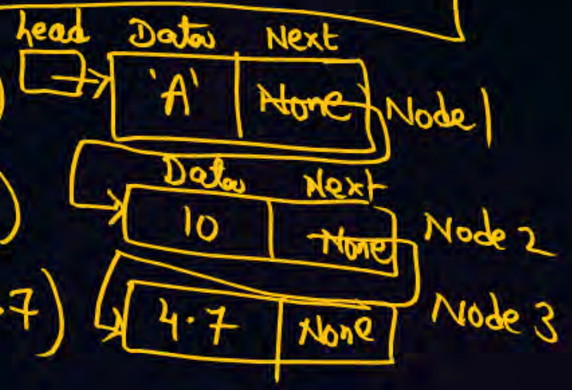
Creation of a SLL Node in Python

- In Python Implementation, Every Node is an object of type SLL class.

```
class SLL :
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
Self.head = Node1
Node1.Next = Node2
Node2.Next = Node3
Node3.Next = None
```

```
Node1 = SLL('A')
Node2 = SLL(10)
Node3 = SLL(4.7)
```



Topic : Linked Lists – Operations, Types

Construct a SLL :

Let SLL Need to be constructed is : (A–D)

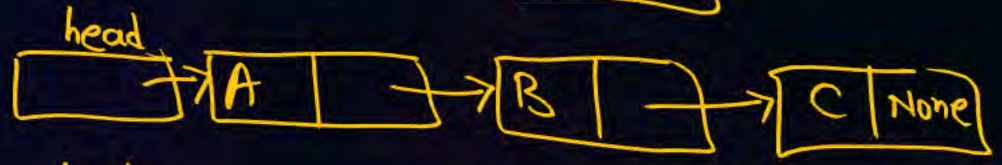
List Construction : Insertion at End ✓

1) Empty list

2) First Node

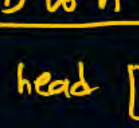
3) Second Node

4) Third Node



5) Fourth Node

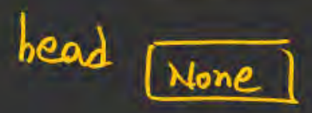
D to A List Construction ≅ Insertion at Beginning



Empty list

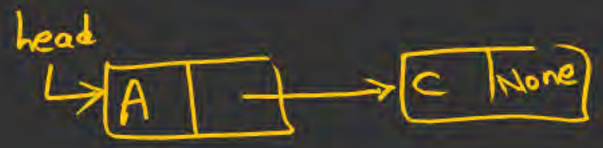
Construction of SLL in random sequence ≡ Insertion of a Node at given Position

Empty List



List with Node C

List with Node C, A

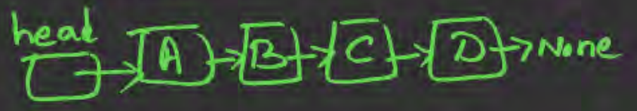


List with Node C, B, A

List with Node D

(Construct a SLL in the sequence A→B→C→D)

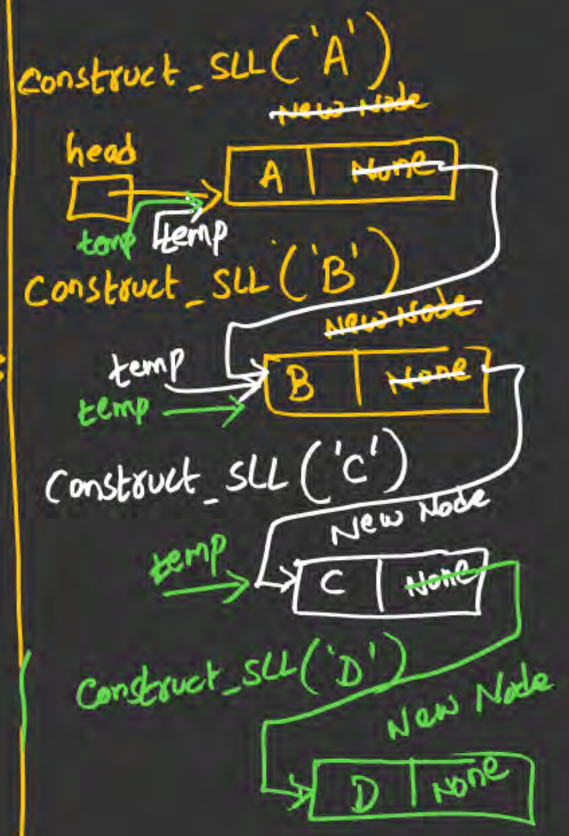
Resultant SLL Constructed:




```
class SLL_Node:
    def __init__(self, data):
        self.data = data
        self.Next = None
```

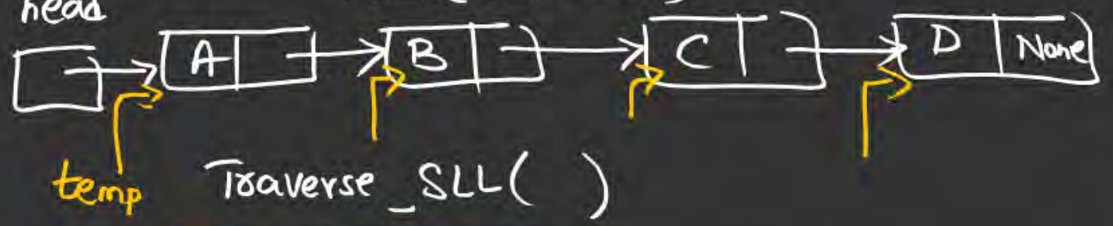
```
class SLL_head:
    def __init__(self):
        self.head = None
```

```
class SLL_Construction:
    def Construct_SLL(value):
        New_Node = SLL_Node(value)
        if self.head is None:   # Empty list
            self.head = New_Node
            New_Node.next = None
        else:
            temp = self.head
            while temp.next is not None:
                temp = temp.next
            temp.next = New_Node
```



Traverse Elements/Nodes in a SLL

```
def Traverse_SLL( ):
    temp = self.head
    while temp.next is not None:
        Print(temp.data, '->')
        temp = temp.next
    Print(temp.data)
```



Traverse_SLL( )

o/p : A->B->C->D

Time Complexity with 'n' nodes : O(n)

Search for a key in SLL

```
def Search_SLL(key):
    temp = self.head
    Status = 0
    while temp.next is not None:
        if temp.data == key:
            Print('key is found')
            Status = 1
            break
        temp = temp.next
    if temp.data != key && Status == 0:
        Print('key is Not found')
```

Time Complexity : O(n)

2 mins Summary

Linked Lists

- Nature
- Types
- operations
- SLL Construction
  - Traverse
  - Search operations.

To be Contd ...

THANK - YOU